
Google News Scraping & Automation Pipeline

Comprehensive Local, Cloud Shell, and Cloud Function Deployment Guide

praXvalue for Opensee

June 2026

Contents

1	Introduction	1
2	Local & Cloud Shell Article Extraction Pipeline	2
2.1	Project Overview	2
2.2	Directory Structure	2
2.3	Environment Setup	2
2.3.1	Install Python Dependencies	3
2.3.2	Install Playwright Browser Binaries	3
2.4	Installation & Permissions	3
2.5	Connection-Triggered Automation	3
2.5.1	How It Works	3
2.6	Manual Execution & Monitoring	3
2.7	Data Pipeline Workflow	4
3	Migration and Running from Another Cloud Shell Account	4
3.1	Step 1: Transfer Project Files	4
3.2	Step 2: Identify and Modify Path References	4
3.2.1	1. Shell Startup Configuration (<code>.bashrc</code>)	5
3.2.2	2. Master Shell Script (<code>run_pipeline.sh</code>)	5
3.2.3	3. Python Scripts	5
3.3	Step 3: Restore Environment and Permissions	5
3.4	Step 4: Enable Connection-Triggered Automation	6

1 Introduction

This document serves as a comprehensive guide for the Google News scraping, article extraction, and automation pipeline designed for **Opensee**.

The pipeline is built as a headless browser-based automation system running in an interactive Linux shell environment (such as Google Cloud Shell). It automates daily scraping routines using connection-triggered execution scripts, ensuring reliable data extraction without relying on a persistent cron daemon.

2 Local & Cloud Shell Article Extraction Pipeline

This section describes the robust, automated pipeline designed for Google Cloud Shell (or any Linux environment) that scrapes Google News articles based on search seeds, resolves their original URLs, and extracts full article text using headless browser automation.

2.1 Project Overview

The local pipeline executes in two sequential stages:

1. **Scraping Stage:** Searches Google News for articles matching a set of search keywords (seeds) from a selected seed file, filtering for articles published in the last 60 days.
2. **Resolution & Extraction Stage:** Uses the Google Custom Search API to resolve original publisher URLs and Playwright (headless Chromium) to render and extract clean article body text (removing headers, footers, scripts, and navigation elements).

The system alternates through **six different seed files** day-by-day in a stateless, round-robin cycle.

2.2 Directory Structure

The project files are structured as follows under the user's home directory. Note that box-drawing characters are represented in standard ASCII format for compatibility across LaTeX compilers:

```
/home/francois_oustry/google_news/
|-- clean_seeds/                # Directory containing seed CSV
    files
|   |-- seeds_buyside_eu_uae.csv
|   |-- seeds_buyside_london.csv
|   |-- seeds_buyside_us.csv
|   |-- seeds_sellside_eu.csv
|   |-- seeds_sellside_london.csv
|   |-- seeds_sellside_us.csv
|-- to_execute_googlenews_opensee_1_0.py # Stage 1: Google News scraper
    script
|-- to_execute_resolveggopensee.py      # Stage 2: URL resolver & text
    extractor script
|-- run_pipeline.sh                    # Master pipeline automation
    script (shell)
|-- pipeline.cron                      # Backup crontab configuration
    file
|-- .last_run_date                     # State file tracking the last
    run date (YYYY-MM-DD)
|-- pipeline_run.log                   # Master execution log file (
    output redirected here)
'-- README.md                          # Markdown reference
    documentation
```

Listing 1: Directory Structure of the Google News Pipeline

2.3 Environment Setup

The pipeline requires **Python 3.8+** and several external dependencies.

2.3.1 Install Python Dependencies

Run the following command in your terminal to install all required libraries:

```
pip3 install gnews pandas playwright beautifulsoup4 requests
```

2.3.2 Install Playwright Browser Binaries

Playwright requires browser binaries to run headless browser instances. Install the Chromium binaries by running:

```
playwright install chromium
```

Note: If running on a bare Linux server instead of Cloud Shell, you may also need to run `playwright install-deps` to install system-level library dependencies.

2.4 Installation & Permissions

To allow the master shell script to run, grant it execution permissions:

```
chmod +x /home/francois_oustry/google_news/run_pipeline.sh
```

2.5 Connection-Triggered Automation

Because Google Cloud Shell VMs shut down automatically after periods of inactivity, a standard crontab is unreliable. Instead, this pipeline uses a **daily login trigger** appended to your shell configuration.

2.5.1 How It Works

Every time you connect to your Cloud Shell terminal, the shell reads your `~/.bashrc` file. The trigger block performs the following steps:

1. Checks if `run_pipeline.sh` exists.
2. Compares today's date with the date in `.last_run_date`.
3. Checks if a pipeline is **already running** in the background using `pgrep`.
4. If a run is needed and not already active:
 - It prints a notification banner.
 - It spawns `run_pipeline.sh` in the background (`&`).
5. **Note:** Unlike previous versions, the trigger does **not** update the `.last_run_date` immediately. Completion is only marked by the script itself upon successful extraction or if it verifies that today's output already exists.

2.6 Manual Execution & Monitoring

You can trigger the entire pipeline manually at any time. It includes a **locking mechanism** (`flock`) that prevents multiple instances from running simultaneously.

```
/home/francois_oustry/google_news/run_pipeline.sh
```

2.7 Data Pipeline Workflow

Below is the step-by-step description of the data flow:

1. Pre-run Checks:

- **Locking:** The script checks `/tmp/google_news_pipeline.lock`. If locked, it exits to prevent concurrent runs.
- **Output Verification:** It checks if a `_fulltext.csv` file for the selected seed and today's date already exists. If found, it marks the day as complete and skips the run.

2. Seed Selection:

Calculates `EPOCH_DAYS` and selects the seed filename via round-robin (`INDEX % 6`).

3. Scraping Stage (`to_execute_googlenews_opensee_1_0.py`):

- Parses the selected seed and queries Google News.
- Saves raw articles as `[seed_base]_[yy-mm-dd-HH].csv`.

4. Extraction Stage (`to_execute_resolveggopensee.py`):

- Resolves URLs and extracts clean text via Playwright.
- Saves final results to `[seed_base]_[yy-mm-dd-HH]_fulltext.csv`.

5. Finalization:

Upon successful completion of all steps, the script updates `.last_run_date` to prevent further triggers today.

3 Migration and Running from Another Cloud Shell Account

To migrate the pipeline and run it from a different Google Cloud Shell account, several configuration and path updates are required. Because the environment paths are tied to the user's home directory structure, running the pipeline under a new username requires transferring the files and adjusting path references.

3.1 Step 1: Transfer Project Files

Ensure all pipeline files and directories are copied to the new Cloud Shell account. The files should be placed under the new user's home directory, preferably in the same subfolder structure:

```
# Example: Creating the directory and copying files
mkdir -p ~/google_news
# Transfer clean_seeds/, Python scripts, and run_pipeline.sh into ~/
google_news/
```

3.2 Step 2: Identify and Modify Path References

All absolute path references to the previous home directory (`/home/francois_oustry/`) must be updated to the new user's home directory (`/home/<new_username>/`).

3.2.1 1. Shell Startup Configuration (.bashrc)

The login-trigger block in ~/.bashrc must be updated. Replace the old paths with the new ones. For robustness and portability across different accounts, it is recommended to use the ~/ or \$HOME environment variables:

```
# In ~/.bashrc, update the path to the runner and state files:
if [ -f "$HOME/google_news/run_pipeline.sh" ]; then
    # ... trigger check logic ...
    # Reference $HOME/google_news/.last_run_date instead of a hardcoded
    # home path
fi
```

3.2.2 2. Master Shell Script (run_pipeline.sh)

Open run_pipeline.sh and inspect the file for any hardcoded references to /home/francois_oustry/. Update these references to use \$HOME or the absolute path of the new home directory:

- Update the working directory path.
- Update the path to the Python executable or virtual environment (if applicable).
- Update paths to the seed files directory (clean_seeds/).

3.2.3 3. Python Scripts

Open the Python script files (to_execute_googlenews_opensee_1_0.py and to_execute_resolveggopensee.py) and verify if there are any hardcoded home directory paths. If absolute paths are used for reading seed files or saving output CSVs, modify them to either:

- Use relative paths (e.g., 'clean_seeds/'), which resolve relative to the script's execution directory.
- Dynamically resolve the home directory in Python using:

```
import os
home_dir = os.path.expanduser('~')
seed_path = os.path.join(home_dir, 'google_news', 'clean_seeds')
```

3.3 Step 3: Restore Environment and Permissions

On the new Cloud Shell account, the dependencies and execution permissions must be re-established:

1. Install Python Libraries:

```
pip3 install gnews pandas playwright beautifulsoup4 requests
```

2. Install Playwright Browser Binaries:

```
playwright install chromium
```

3. Make Master Script Executable:

```
chmod +x ~/google_news/run_pipeline.sh
```

3.4 Step 4: Enable Connection-Triggered Automation

Once the paths are updated and permissions are set, append the daily login trigger block to the new account's `~/.bashrc` file to reactivate daily background automation upon login.